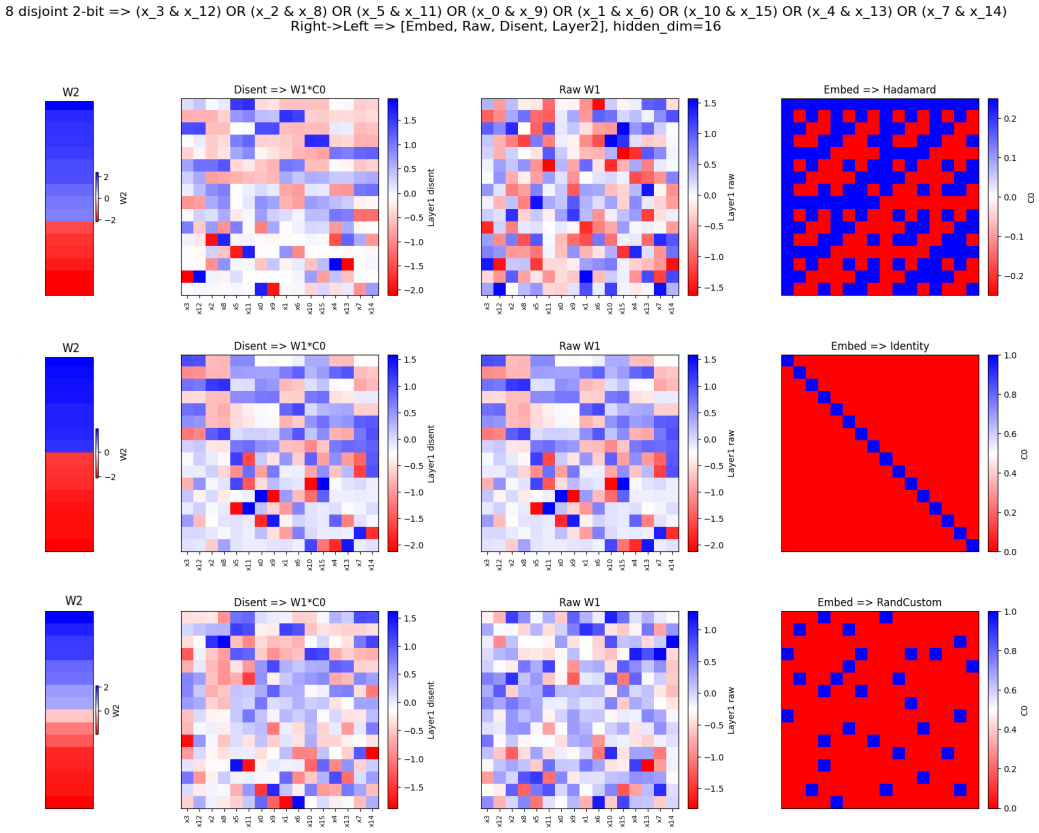

Figure 22: Disentangling of networks computing the feature circuit depicted in Figure 21. In top a 2-layer network with an initial Hadamard embedding layer C_0 that is trained as before on a DNF of 8 different 2-AND clauses. The raw trained W_1 matrix is shown immediately to the left of it. Then to its left we show the disentangled $C_1 = W_1 C_0$. Notice that C_1 has the usual expected structure with coding of 2P in the positive witness rows and coding for a XOR using 1N1P in the negative witness rows. Moreover, though having less 2P patterns and positive rows, it has a similar structure as the case where there is no embedding (the embedding is the identity matrix) as shown in the middle row of the figure. The bottom row shows how a symmetric random embedding is disentangled in a similar fashion.

in the disentangled space would give us the insights that are missing when using activation space approaches.

In the above experiments, we have used three different types of embeddings: identity (or no embedding), using a Hadamard matrix, and using a symmetric random embedding. In most of the statistics in Section 4 we trained on a network where the input had a Hadamard embedding layer C_0 which we then used to get $W_1 * C_0$, the coding matrix in which we counted the number of patterns (as a sanity check, we also ran the same training without the Hadamard embedding, getting similar results). In all other sections we used the identity embedding.

Finally, one can understand today's prevailing approach to interpreability via the training of an autoencoder [29] using our coding-decoding view of computation. The autoencoder approach is a way of learning an approximation of D_i for a given layer i. The autoencoder has a very large inner layer to which the output activations of C_{i-1} are mapped, along with a sparsity constraint to separate them so that the features of layer $i-1$ can be read. The difference in the combinatorial approach we propose here is to attempt to decode C_{i-1} and reconstruct D_i by analyzing the network weight matrices themselves rather then learning features from the activations using a separate trained network. As mentioned above, we think there is potential in a combined approach, for example where D_i^* is learned via an autoencoder (or other activation based learning approaches [10]), and then this is used to find and interpret C_i combinatorially. This would allow our approach to be applied to a single layer, without the need for cascading.

7 Towards Deeper Networks

As a bit of a teaser as to the potential directions one can follow with the combinatorial interpretability approach, we examine what happens if there are multiple outputs - i.e., a classification problem with more than 2 possibilities. To do so, we add additional Boolean formulas to learn, and for each additional formula, there is a corresponding additional second layer output neuron responsible for that formula. The inputs and the first hidden layer are shared between the different outputs, and so the output neurons are computing different formulas on the same set of Boolean variables and on the same hidden layer results. We still only consider the case where there is a single hidden layer, but we view increasing the size of the output layer as a small step towards having an additional hidden layer, as view computing multiple output features as a potential proxy for computing multiple features for use at the next hidden layer of the network.

We first consider the case with two output neurons. We tested a range of clause numbers, and found something similar to the single output case: the system is able to train fairly successfully, provided that the total number of clauses (in this case, summed across all outputs) is not larger than the hidden dimension. We here provide results for one representative example: in this case there are 32 Boolean variables and a hidden dimension of 32. The formulas are in DNF form with clauses of size 4 (4-AND), and each output is the OR of 8 clauses. The clauses for each output are non-overlapping in terms of variables, but the clauses for one output are chosen without taking into account the other output. Thus, each variable is used exactly once in each output but can be used in both outputs.

We generated 50000 inputs, each having between 8 and 10 variables set to true. The inputs were constructed independently, and each had equal likelihood of being each of the 4 output combinations (00, 01, 10, 11). Each output possibility was constructed in the same style as used in Section 4.

We provide the full neural network parameterization for the following pair of formulas:

$$\begin{aligned} f_0(x) = {} & (x_3 \wedge x_4 \wedge x_7 \wedge x_{10}) \vee (x_2 \wedge x_6 \wedge x_{25} \wedge x_{27}) \vee (x_0 \wedge x_{13} \wedge x_{30} \wedge x_{31}) \vee \\ & (x_9 \wedge x_{16} \wedge x_{20} \wedge x_{21}) \vee (x_8 \wedge x_{12} \wedge x_{14} \wedge x_{28}) \vee (x_1 \wedge x_5 \wedge x_{17} \wedge x_{24}) \vee \\ & (x_{11} \wedge x_{22} \wedge x_{26} \wedge x_{29}) \vee (x_{15} \wedge x_{18} \wedge x_{19} \wedge x_{23}) \end{aligned}$$

$$f_1(x) = (x_{10} \wedge x_{15} \wedge x_{23} \wedge x_{26}) \vee (x_9 \wedge x_{13} \wedge x_{18} \wedge x_{19}) \vee (x_0 \wedge x_4 \wedge x_8 \wedge x_{14}) \vee$$
$$(x_6 \wedge x_{20} \wedge x_{28} \wedge x_{31}) \vee (x_3 \wedge x_{21} \wedge x_{22} \wedge x_{30}) \vee (x_2 \wedge x_{11} \wedge x_{12} \wedge x_{24}) \vee$$
$$(x_1 \wedge x_5 \wedge x_{25} \wedge x_{29}) \vee (x_7 \wedge x_{16} \wedge x_{17} \wedge x_{27})$$

This computation is depicted in Figure 23. This network achieved training loss of 3.42% after 1000 epochs, and at that time had overall accuracy (both bits correct) on the test set of 97.30%. Not only did the network learn this network very clearly, we see clear fature channel coding, despite having to do that coding for two different Boolean functions simultaneously.

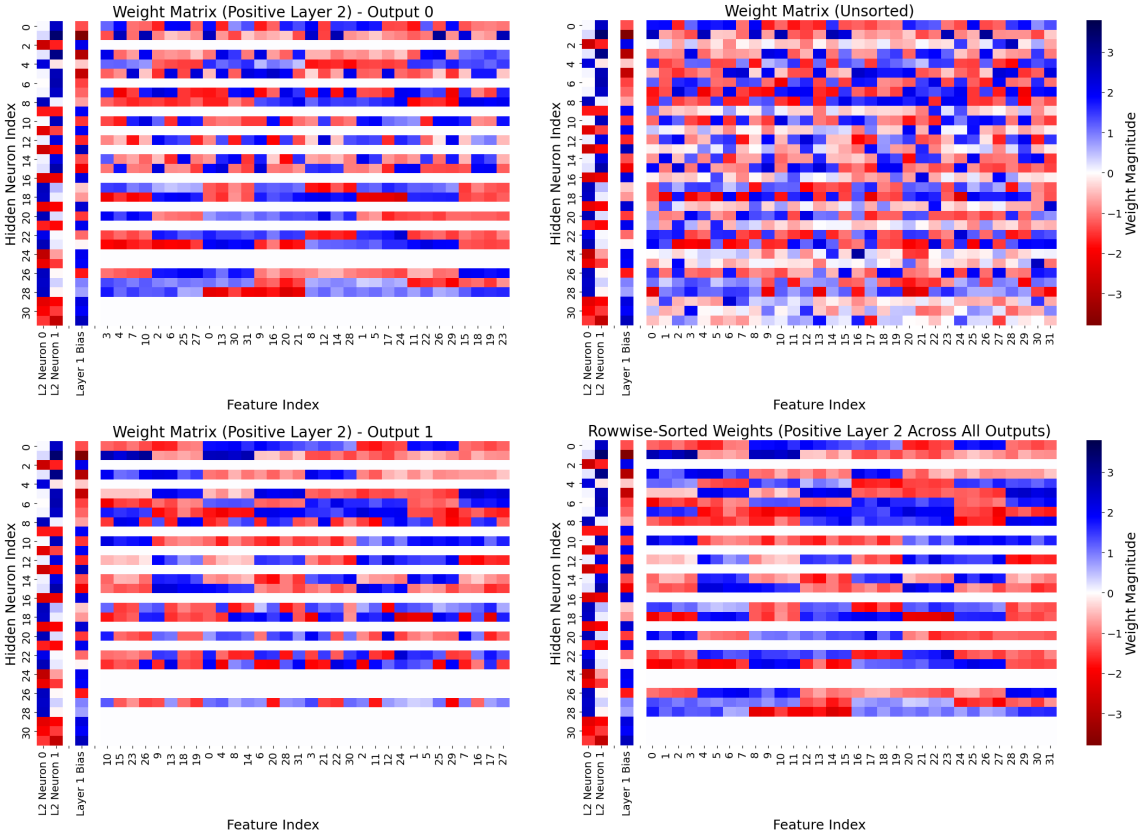

Figure 23: Trained neural network for f_0 and f_1. The top right is the unsorted version of the weight matrix, with all weights and biases. The top left is sorted by the clauses of f_0 (computed by output 0), and, as above, filtered to show only those neurons that have a positive edge to output neuron 0. The bottom left is the equivalent version for output neuron 1 and f_1. The bottom right has all rows with a positive edge to either output neuron, and each such row is sorted by the clauses corresponding to the output neuron with the larger weight to that row/hidden layer neuron.

A few observations about this parameterization: